

1 Recursão

A ideia de qualquer algoritmo recursivo para um problema é igual à de uma prova por indução matemática:

1. se a instância que queremos resolver é pequena, resolva-o diretamente, como puder.
2. se a instância é grande, reduza-a a uma instância menor do mesmo problema.
3. depois de resolvida a instância menor, volte à instância original.

Para que a ideia da recursão possa ser aplicada é preciso que o problema tenha uma "estrutura recursiva".

1.1 Exemplo: Busca Sequencial Recursiva

Vamos analisar o problema da busca sequencial e faremos uma solução recursiva para o problema. O problema da busca sequencial pode ser definido da seguinte maneira:

- Dado um conjunto de elementos $A = \{s_1, s_2, \dots, s_n\}$ e um elemento alvo x o objetivo é determinar se o elemento x está presente no conjunto A , e em caso afirmativo o algoritmo deve retornar sim, se não o retorno é não.
- **Entradas:**
 - Um conjunto de elementos $A = \{s_1, s_2, \dots, s_n\}$
 - O tamanho do conjunto A de nome n
 - Um elemento alvo x
- **Retorno:**
 - Sim, se o elemento estiver no conjunto.
 - Não, se o elemento não estiver presente no conjunto

Algorithm 1 Busca Sequencial Recursiva

```
1: function BUSCA( $A, n, x$ )
2:   if  $n = 0$  then
3:     return NÃO                                     ▷ Elemento não encontrado
4:   end if
5:   if  $A[n] = x$  then
6:     return SIM                                     ▷ Elemento encontrado
7:   else
8:     return BUSCA( $A, n - 1, x$ )                     ▷ Chama a função Busca recursivamente
9:   end if
10: end function
```

Base

Primeiro vamos definir $P(n)$ como $BUSCA(A, n, x)$ que devolve SIM se $x \in A$ e devolve NÃO se $x \notin A$

- Se $n = 0$, o vetor está vazio, e o algoritmo retorna NÃO. Isso é correto, pois não há elementos para verificar. Logo $P(0) = \text{NÃO}$.

Hipótese

- $P(k)$ vale, para $0 \leq k < n$, onde $P(k)$ devolve SIM se $x \in A$ e devolve NÃO se $x \notin A$

Indução

- Quando o algoritmo recebe $n > 0$ ele verifica se $A[n] = x$. Se forem iguais, devolve SIM, portanto o algoritmo $P(n)$ é válido nesse caso.
- Se $A[n] \neq x$, ele devolve o mesmo de chamada $P(k)$, onde $k = n - 1$, mas por hipótese essa chamada corretamente detecta se $x \in A[n, \dots, k]$. como já sabemos que $A[n] \neq x$, concluímos que a chamada atual funciona.

Portanto, o algoritmo BUSCA é correto, pois verifica todos os elementos do vetor e retorna a resposta correta com base na presença ou ausência de x .

2 Recorrências

Para analisar o consumo de tempo de um algoritmo recursivo é necessário resolver uma recorrência. Uma recorrência (= recurrence) é uma expressão que dá o valor de uma função num dado ponto em termos dos valores da mesma função em pontos anteriores. Podemos analisar a recorrência do algoritmo de busca sequencial recursivo visto anteriormente

$$T(n) = \begin{cases} O(1) & \text{se } n = 0 \\ T(n-1) + O(1) & \text{se } n > 0 \end{cases}$$

- $T(n)$ é o número de operações realizadas para um vetor de tamanho n .
- $O(1)$ é o número de operações para o caso base (quando $n = 0$)
- $O(1)$ é o número de operações para cada passo recursivo.

2.1 Resolvendo recorrências

Resolver uma recorrência é encontrar uma fórmula fechada que dê o valor da função diretamente em termos de seu argumento (e sem subexpressões da forma $+\dots+$ ou contendo $\sum$ ou $\prod$). Tipicamente, a fórmula fechada é uma combinação de polinômios, quocientes de polinômios, logaritmos, exponenciais, etc.

2.1.1 Método da substituição

O método da substituição consiste em provar por indução que $T(n)$ é limitada por $f(n)$, vamos analisar melhor como funciona usando um exemplo.

1. Exemplo:

$$T(n) = \begin{cases} T(1) = 1 & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{se } n > 1 \end{cases}$$

Queremos provar que $T(n) = O(n^2)$. Aqui estou chutando que o algoritmo é $O(n^2)$

$$2T\left(\frac{n}{2}\right) + n \leq c \cdot n^2$$

Vamos provar por indução que $T(n) \leq c \cdot n^2$.

- **Base:** Se $n = 1$, sabemos que $T(1) = 1$ e $c \cdot 1^2 = c$. Então vale se $c \geq 1$.
- **Hipótese:** $T(k) \leq ck^2$ para $1 \leq k < n$.
- **Indução:** Sabemos que $T(n) = 2T\left(\frac{n}{2}\right) + n$. Como $\frac{n}{2} < n$ (pois $n > 1$), vale por hipótese que $T\left(\frac{n}{2}\right) \leq c \cdot \left(\frac{n}{2}\right)^2 = \frac{cn^2}{4}$. Então:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &\leq 2 \left[\frac{cn^2}{4} \right] + n \\ &= \frac{cn^2}{2} + n \\ &\leq cn^2 \end{aligned}$$

Note que $\frac{cn^2}{2} + n \leq cn^2$ se $n \leq \frac{cn^2}{2}$, ou $1 \leq \frac{cn}{2}$, ou $c \geq \frac{2}{n}$. Como $\frac{2}{n} < 2$ sempre que $n > 1$, basta que $c \geq 2$.

2.1.2 Método da iteração

Consiste em expandir a recorrência até chegar no caso base, que sabemos calcular diretamente.

1. Exemplo:

$$T(n) = \begin{cases} T(1) = 1 & \text{se } n = 1 \\ 2T(n-1) + n & \text{se } n > 1 \end{cases}$$

Vamos começar expandindo a função.

$$T(n) = 2[T(n-1)] + n \tag{1}$$

$$= 2[2T(n-2) + n-1] + n = 4[T(n-2)] + 3n-2 \tag{2}$$

$$= 4[2T(n-3) + n-2] + 3n-2 = 8[T(n-3)] + 7n-10 \tag{3}$$

$$= 8[2T(n-4) + n-3] + 7n-10 = 16T(n-4) + 15n-31 \tag{4}$$

Agora vamos tentar definir como essa função cresce para a i -ésima iteração da função para encontrarmos a fórmula fechada "achando o padrão" na função.

Não vou entrar nos detalhes matemáticos para encontrar a função em valor em i para simplificar a explicação, mas é fácil entender como foi achado observando a função em i e as funções nas iterações iniciais da função.

i	$T(n_i)$
1	$2[T(n-1)] + n$
2	$4[T(n-2)] + 3n$
3	$8[T(n-3)] + 7n - 10$
4	$16T(n-4) + 15n - 31$
i	$2^i T(n-i) + (2^i - 1)n + \sum_{j=1}^{i-1} 2^j(-j)$

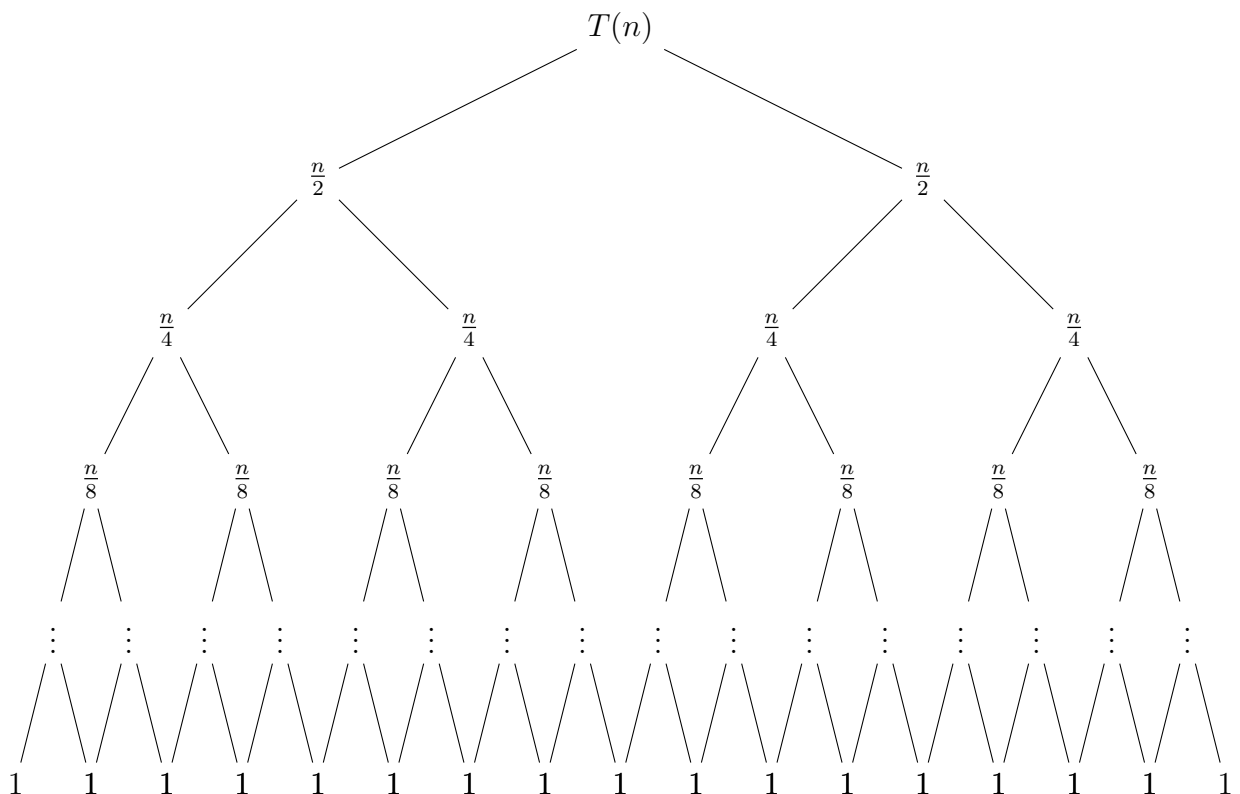
Agora que temos uma fórmula fechada para a i -ésima iteração da função, podemos realizar quaisquer operações com ela, inclusive calcular os limites inferiores e superiores da função.

2.1.3 Método da árvore de recursão

O método consiste em analisar a árvore de recursão onde cada nó representa um subproblema (chamada recursiva), um nó é rotulado com o tempo gasto naquela chamada e os filhos do nó são as chamadas recursivas que ele fez por ele. Então se obtivermos o tempo gasto em um nível e somarmos os tempos de todos os níveis, temos uma estimativa total de tempo.

1. Exemplo:

$$T(n) = \begin{cases} T(1) = 1 & \text{se } n = 1 \\ 2T(\frac{n}{2}) + n & \text{se } n > 1 \end{cases}$$



Observação: Coloquei que o o último nível da árvore com um formato "estranho" para polpar mais espaço, mas imagine que cada nó do nível "...tem dois filhos e cada um desses filho tem o valor 1

Agora vamos fazer uma tabela para analisar a árvore

Nível (i)	Tamanho da entrada	Nós	Tempo por nó
1	n	1	n
2	$\frac{n}{2}$	2	$\frac{n}{2}$
3	$\frac{n}{4}$	4	$\frac{n}{4}$
4	$\frac{n}{8}$	8	$\frac{n}{8}$
i	$\frac{n}{2^i}$	2^i	$\frac{n}{2^i}$
?	1	$2^?$	1

Para determinar o número de níveis da sua árvore de recursão, você pode usar a seguinte abordagem. Sabemos que o tamanho do problema no último nível deve ser 1, o que pode ser representado como $\frac{n}{2^?}$, onde ? é o número de níveis na árvore. Portanto, para encontrar o valor de ?, precisamos resolver a equação:

$$\frac{n}{2^?} = 1$$

Isso implica que $n = 2^?$, pois quando 1 é dividido por $2^?$, o resultado é 1. Assim, para determinar o número de níveis (?), precisamos encontrar o expoente ao qual 2 deve ser elevado para igualar n . Isso pode ser feito tomando o logaritmo base 2 de n . Então, o número de níveis ? é dado por:

$$? = \log_2(n)$$

Essa é uma maneira eficiente de determinar o número de níveis da sua árvore de recursão, utilizando a propriedade de que o tamanho do problema no último nível é 1.

Para seguir a lógica vou reescrever a tabela substituindo ? por $\log_2(n)$

Nível (i)	Tamanho da entrada	Nós	Tempo por nó
1	n	1	n
2	$\frac{n}{2}$	2	$\frac{n}{2}$
3	$\frac{n}{4}$	4	$\frac{n}{4}$
4	$\frac{n}{8}$	8	$\frac{n}{8}$
i	$\frac{n}{2^i}$	2^i	$\frac{n}{2^i}$
$\log_2(n)$	1	n	1

Observe que o número de nós no nível $\log_2(n)$ é $2^{\log_2(n)}$, o que simplifica para n , utilizando a propriedade do expoente logarítmico.

Agora que sabemos que temos $\log_2(n)$ níveis, que cada nível tem 2^i nós e que o custo por nível é $\frac{n}{2^i}$, podemos fazer o somatório dos custos de cada um dos nós da minha árvore para descobrir o tempo de execução do algoritmo. Logo, o tempo total da árvore pode ser escrito como:

$$\sum_{i=0}^{\log_2(n)} \frac{n}{2^i} \cdot 2^i$$

A lógica para esta solução é simples: para cada nível, multiplicamos o custo por nó ($\frac{n}{2^i}$) pela quantidade de nós no nível (2^i), e assim chegamos a este somatório. Para esse caso, podemos continuar a simplificação.

$$\begin{aligned} & \sum_{i=0}^{\log_2(n)} \frac{n}{2^i} \cdot 2^i \\ &= \sum_{i=0}^{\log_2(n)} n \\ &= n(\log_2(n) + 1) \\ &= n \log_2(n) + n \end{aligned}$$

Assim chegamos a uma fórmula fechada para a recorrência usando o método da árvore de recursão.

2.1.4 Teorema mestre

O método mestre pode ser utilizado para resolver recorrências no formato:

$$T(n) = aT(n/b) + f(n)$$

onde $a \geq 1$ e $b \geq 1$ e a tanto a quanto b são constantes e $f(n)$ é assintoticamente positiva. O caso base é omitido na definição e convencionou-se que é uma constante para valores pequenos. É válido mencionar que a expressão $\frac{n}{b}$ pode indicar tanto o $\lfloor \frac{n}{b} \rfloor$ quanto o $\lceil \frac{n}{b} \rceil$.

Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ definida para os inteiros não negativos pela relação de recorrência

$$T(n) = aT(n/b) + f(n).$$

Então $T(n)$ tem os seguintes limitantes assintóticos:

1. Se $f(n) \in O(n^{\log_b a - \epsilon})$ para alguma constante $\epsilon > 0$, então $T(n) \in \Theta(n^{\log_b a})$.
2. Se $f(n) \in \Theta(n^{\log_b a})$, então $T(n) \in \Theta(n^{\log_b a} \log n)$.
3. Se $f(n) \in O(n^{\log_b a + \epsilon})$ para alguma constante $\epsilon > 0$ e se $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e todo n suficientemente grande, então $T(n) \in \Theta(f(n))$.

Vamos resolver um exemplo para cada um dos casos do teorema mestre.

1. **Exemplo:** $T(n) = 9T(n/3) + n$

Aqui vamos usar o primeiro caso do teorema mestre. Primeiro, vamos definir $a = 9$, $b = 3$ e $f(n) = n$. Calculando o $\log_b a$:

$$\log_b a = \log_3 9 = 2.$$

Como $f(n) = O(n^{\log_3 9 - \epsilon})$ é positivo para $\epsilon = 1$ e como $n^{\log_3 9} = 2$, pelo item 1 do teorema mestre, temos que $T(n) = \Theta(n^2)$.

2. **Exemplo:** $T(n) = T(\frac{2n}{3}) + 1$ Definimos $a = 1$, $b = \frac{2}{3}$ e $f(n) = 1$. Calculando o $\log_b a$:

$$\log_b a = \log_{\frac{2}{3}} 1 = 0.$$

Logo,

$$f(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_{\frac{2}{3}} 1}) = \Theta(n^0) = \Theta(1).$$

Substituindo no caso 2 do teorema mestre:

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a} \cdot \log n) \\ &= \Theta(n^{\log_{\frac{2}{3}} 1} \cdot \log n) \\ &= \Theta(1 \cdot \log n) \\ &= \Theta(\log n). \end{aligned}$$

3. **Exemplo:** $T(n) = 3T(n/4) + n \log n$

Temos definido $a = 3$, $b = 4$ e $f(n) = n \log n$. Vamos iniciar calculando o $\log_b a$:

$$\log_b a = \log_4 3 = \frac{\log 3}{\log 4} \approx 0.79248 \dots$$

Logo, $n^{\log_4 3} = n^{0.79248 \dots}$. Fazendo $\epsilon = 1 - 0.79248 \dots$, temos que $f(n) = \Omega(n^{\log_4 3 + \epsilon})$. Como $aF(\frac{n}{b}) = 3(\frac{n}{4} \log \frac{n}{4}) = \frac{3}{4}(\log n - \log 4) \leq \frac{3}{4}n \log n \leq \frac{3}{4}f(n)$, podemos considerar $c = \frac{3}{4}$. Portanto, pelo item 3 do teorema mestre, $T(n) = \Theta(n \log n)$.

3 Correções e Comentários

Encontrou algum erro? Por favor, entre em contato pelo email hericsilvaho@gmail.com com os detalhes pertinentes. Agradeço a sua contribuição!